METRIK *TAINT-GROUNDED* UNTUK EVALUASI XAI PADA DETEKSI SQLI JAVA

Disusun sebagai syarat kelulusan mata kuliah
IF4091/Penyusunan Proposal

Oleh

Naufarrel Zhafif Abhista
NIM: 13523149



PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Oktober 2026

METRIK *TAINT-GROUNDED* UNTUK EVALUASI XAI PADA DETEKSI SQLI JAVA

Oleh

Naufarrel Zhafif Abhista
NIM: 13523149

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Bandung, 21 Oktober 2026

Mengetahui,

Pembimbing I,	Pembimbing II,
Ir. Yudistira Dwi Wardhana Asnar, S.T, Ph.D.	Latifa Dwiyanti, S.T., M.T.
NIP.	NIP.

DAFTAR ISI

BAB I PENDAHULUAN 1
I.1 Latar Belakang 1
I.2 Rumusan Masalah 1
I.3 Tujuan dan Ukuran Keberhasilan Pencapaian 1
I.4 Batasan Masalah 1
I.5 Metodologi 2

BAB II KAJIAN PUSTAKA 3
II.1 *SQL Injection* dan *Taint Analysis* 3
II.1.1 *SQL Injection* sebagai Kerentanan Aliran Data 3
II.1.2 *Taint Analysis* dan Perangkat Analisis 4
II.2 Deteksi Kerentanan Berbasis *Machine Learning* 4
II.3 *Explainable AI* untuk Model Kode 5
II.3.1 Konsep Dasar XAI 5
II.3.2 SHAP dan LIME 6
II.3.3 *Attention* sebagai Penjelasan 6
II.4 Evaluasi Penjelasan XAI 7
II.4.1 Keterbatasan Metrik yang Ada untuk Domain Kode 7
II.5 Penelitian Terkait dan Celah Penelitian 7
II.5.1 Celah Penelitian 7

BAB III ANALISIS MASALAH DAN RANCANGAN SOLUSI 8
III.1 Analisis Masalah 8
III.2 Gambaran Umum Solusi 8

BAB IV RENCANA PELAKSANAAN 9
IV.1 Jadwal 9
IV.2 Risiko 9

DAFTAR PUSTAKA 10

DAFTAR LAMPIRAN

Lampiran A. Contoh Judul Lampiran 12
A.1 Contoh Judul Anak Lampiran 12

DAFTAR GAMBAR

DAFTAR TABEL

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
SQLi	*SQL Injection*	...
XAI	*Explainable Artificial Intelligence*	...

DAFTAR ISTILAH

Istilah	Deskripsi
<istilah>	<penjelasan istilah>

BAB I
PENDAHULUAN

Bab Pendahuluan secara umum yang dijadikan landasan kerja dan arah kerja penulis tugas akhir, berfungsi mengantar pembaca untuk membaca laporan tugas akhir secara keseluruhan.

I.1 Latar Belakang

Latar Belakang berisi dasar pemikiran, kebutuhan atau alasan yang menjadi ide dari topik tugas akhir. Tujuan utamanya adalah untuk memberikan informasi secukupnya kepada pembaca agar memahami topik yang akan dibahas. Saat menuliskan bagian ini, posisikan anda sebagai pembaca – apakah anda tertarik untuk terus membaca?

I.2 Rumusan Masalah

Tuliskan rumusan masalah yang menjadi dasar perlunya tugas akhir dibuat. Rumusan masalah yang baik memiliki struktur sebagai berikut:

1. Diawali dengan ringkasan dalam satu atau dua kalimat tentang kondisi/situasi dalam bentuk *problem statement* yang ada sekarang terkait dengan topik utama yang dibahas pada tugas akhir.
2. Pokok persoalan dari kondisi/situasi yang ada, dapat dilihat dari kelemahan atau kekurangannya. Bagian ini merupakan inti dari rumusan masalah.
3. Elaborasi lebih lanjut yang menekankan hal-hal penting sebagai pokok kajian untuk menyelesaikan persoalan tugas akhir.
4. Dapat ditambahkan usulan singkat terkait dengan solusi yang ditawarkan untuk menyelesaikan persoalan tugas akhir.

Penting untuk diperhatikan bahwa persoalan yang dideskripsikan pada subbab ini akan dipertanggungjawabkan di akhir pelaksanaan tugas akhir, apakah terselesaikan atau tidak?

I.3 Tujuan dan Ukuran Keberhasilan Pencapaian

Tuliskan tujuan utama dan/atau tujuan detil yang akan dicapai dalam pelaksanaan tugas akhir. Fokuskan pada hasil akhir yang ingin diperoleh setelah tugas akhir diselesaikan, terkait dengan penyelesaian persoalan pada rumusan masalah. Penting untuk diperhatikan bahwa tujuan yang dideskripsikan pada subbab ini akan dipertanggungjawabkan di akhir pelaksanaan tugas akhir apakah tercapai atau tidak. Tuliskan juga ukuran keberhasilannya.

I.4 Batasan Masalah

Tuliskan batasan-batasan kajian yang diambil dalam pelaksanaan tugas akhir dan bukan merupakan batasan implementasi solusi. Tuliskan batasan yang dicakup tersebut dalam kalimat positif. Batasan ini dapat dihindari (tidak perlu ada) jika topik/judul tugas akhir dibuat cukup spesifik.

I.5 Metodologi

Tuliskan semua tahapan yang akan dilalui selama pelaksanaan tugas akhir. Tahapan ini spesifik untuk menyelesaikan persoalan tugas akhir. Tahapan studi literatur tidak perlu dituliskan karena ini adalah pekerjaan yang harus Anda lakukan selama proses pelaksanaan tugas akhir.

BAB II
KAJIAN PUSTAKA

Bab ini memaparkan dasar teori dan penelitian terkait yang melandasi pengembangan metrik *taint-grounded* untuk mengevaluasi penjelasan XAI pada deteksi *SQL injection* di Java. Struktur pembahasan dimulai dari kerentanannya, yakni *SQL injection*, hingga ke cara menilai penjelasan atas kerentanan tersebut.

II.1 *SQL Injection* dan *Taint Analysis*

II.1.1 *SQL Injection* sebagai Kerentanan Aliran Data

SQL Injection adalah sebuah kerentanan (*vulnerability*) yang terjadi ketika ada masukan dari pengguna yang dianggap dan diperlakukan sebagai kode SQL yang valid (Halfond, Viegas, dan Orso, 2006). Penyebab utama dari kerentanan ini adalah kurangnya validasi masukan pengguna (Halfond dkk., 2006).

Listing II.1. Contoh kode Java yang rentan terhadap *SQL injection*

```
1 String id = request.getParameter("id");
2 Statement stmt = conn.createStatement();
3 ResultSet rs = stmt.executeQuery(
4     "SELECT * FROM users WHERE id = '" + id + "'");
```

Pada aplikasi web, masukan tersebut umumnya berasal dari isian formulir yang dikirim melalui permintaan HTTP GET atau POST (Halfond dkk., 2006). Pola yang rentan pada Java muncul ketika nilai masukan digabungkan langsung ke teks *query* dengan *string concatenation*, lalu dieksekusi, seperti pada Listing II.1. Pola serupa dipakai Halfond dkk. (2006) sebagai contoh implementasi *servlet* yang rentan. Pada Listing II.1, nilai `id` diambil dari parameter permintaan pada baris 1, digabungkan ke teks *query* pada baris 4, dan diteruskan ke `executeQuery` pada baris 3–4.

Bila penyerang mengirim `1' OR '1'='1` sebagai `id`, *query* yang terbentuk menjadi `SELECT * FROM users WHERE id = '1' OR '1'='1'`, yang kondisinya selalu benar sehingga seluruh baris tabel dikembalikan. Serangan jenis ini dikenal sebagai tautologi, yaitu menyisipkan kode ke kondisi sehingga selalu bernilai benar (Halfond dkk., 2006).

Jika ditinjau dari aliran datanya, *SQL injection* terjadi ketika masukan dari pengguna mengalir sampai ke perintah yang mengeksekusi *query* tanpa dibersihkan terlebih dahulu. Pada Listing II.1, masukan masuk melalui `getParameter` dan sampai di `executeQuery` tanpa ada validasi di antara keduanya (kedua titik ini dinamai *source* dan *sink* pada Subbab II.1.2).. Aliran data ini dapat ditelusuri menggunakan pendekatan analisis *taint*. Salah satu contohnya, WebSSARI, memakai *static analysis* untuk memeriksa apakah data *tainted* sampai ke fungsi yang sensitif (misalnya yang mengeksekusi *query*) (Halfond dkk., 2006). Berdasarkan hal tersebut, *taint analysis* (Subbab II.1.2) dapat dipakai untuk menunjukkan bagian kode yang menjadi penyebab kerentanan.

II.1.2 *Taint Analysis* dan Perangkat Analisis

Taint analysis adalah teknik untuk menelusuri perambatan (propagasi) data di dalam program (Yamaguchi, Maier, Gascon, dan Rieck, 2015). Tujuannya adalah menemukan aliran data dari *source* yang dikendalikan penyerang menuju *sink* yang sensitif tanpa melalui proses sanitasi (*sanitization*) (Yamaguchi dkk., 2015). Sanitasi adalah pembersihan atau pemeriksaan masukan sebelum dipakai, misalnya dengan melakukan *escape*, pemeriksaan tipe, dan penyaringan (Halfond dkk., 2006). Untuk menjalankannya, analisis harus lebih dulu menentukan *source*, *sink*, dan aturan sanitasi yang berlaku (Yamaguchi dkk., 2015).

Pada Listing II.1, `getParameter` berperan sebagai *source* dan `executeQuery` sebagai *sink*, tanpa ada sanitasi di antara keduanya. Kerentanan dengan pola seperti ini disebut kerentanan bergaya *taint* (*taint-style vulnerabilities*), dan *SQL injection* termasuk di dalamnya (Yamaguchi dkk., 2015). Pada proposal ini, lintasan aliran data dari *source* ke *sink* tersebut disebut *taint-flow path*.

Yamaguchi, Golde, Arp, dan Rieck (2014) menyatakan bahwa banyak kerentanan hanya dapat ditemukan dengan memperhatikan struktur, aliran kendali, dan ketergantungan kode secara bersamaan. Untuk itu, mereka memperkenalkan *code property graph* (CPG), yaitu graf yang menggabungkan tiga representasi klasik dalam satu struktur data (Yamaguchi dkk., 2014):

- *Abstract syntax tree* (AST), yang menunjukkan bagaimana pernyataan dan ekspresi saling bersarang membentuk program (Yamaguchi dkk., 2014).
- *Control flow graph* (CFG), yang menggambarkan urutan eksekusi pernyataan beserta kondisi yang harus dipenuhi untuk menempuh suatu jalur (Yamaguchi dkk., 2014).
- *Program dependence graph* (PDG), yang memuat ketergantungan data (pengaruh satu variabel terhadap variabel lain) dan ketergantungan kendali (pengaruh kondisi terhadap nilai variabel) (Yamaguchi dkk., 2014).

Dengan CPG, pencarian aliran data tanpa sanitasi dapat dinyatakan sebagai penelusuran (*traversal*) pada graf (Yamaguchi dkk., 2015). Salah satu perangkat yang menerapkan CPG adalah Joern, yaitu platform analisis kode *open source* (Baker Effendi, Pinho, Dreyer, dan Yamaguchi, 2025). Melalui *language frontend*-nya, Joern menghasilkan AST, CFG, dan *control dependence graph* (CDG) terpadu yang membentuk CPG hampir lengkap (Baker Effendi dkk., 2025). Implementasi awal CPG memakai *parser* C/C++ (Yamaguchi dkk., 2014), sedangkan efisiensi analisis aliran data pada Joern kini telah dievaluasi pada program Java, Python, dan JavaScript (Baker Effendi dkk., 2025). Sebagai pembanding, Baker Effendi dkk. (2025) menggunakan Semgrep dan CodeQL, yaitu perangkat analisis statis yang luas dipakai, mendukung analisis *taint* multi-bahasa, dan dapat menganalisis program parsial (Baker Effendi dkk., 2025).

II.2 Deteksi Kerentanan Berbasis *Machine Learning*

Model *machine learning* memerlukan kode sumber dalam bentuk representasi vektor sebagai masukan (Ganz, Rall, Härterich, dan Rieck, 2023). Salah satu caranya adalah memperlakukan kode seperti teks, yaitu pernyataan dianggap sebagai kalimat, sedangkan kata kunci dan literal dianggap sebagai kata (Ganz dkk., 2023). Cara lain

adalah memodelkan kode sebagai graf yang dapat menangkap hubungan sintaktis dan semantik antarpernyataan dan ekspresi (Ganz dkk., 2023). Luo, Xu, dan Xu (2022) menyatakan bahwa representasi kode sangat penting bagi pendekatan deteksi kerentanan berbasis *machine learning*. Graf ringkas yang mereka usulkan (*compact abstract graphs*) menghasilkan model GNN yang secara signifikan lebih baik daripada AST, CFG, dan PDG, dengan akurasi 94,7% sampai 96,3% pada dataset Java berisi 114 jenis kerentanan (Luo dkk., 2022).

Dari sisi model, CodeBERT adalah model *pre-trained* bimodal untuk bahasa pemrograman dan bahasa alami yang dibangun dengan arsitektur *Transformer* (Feng dkk., 2020).

Meskipun metode deteksi berbasis *deep learning* menunjukkan performa kuat pada dataset *benchmark*, efektivitasnya di dunia nyata masih kurang dieksplorasi (Lu dan Lagaisse, 2025). Dataset *benchmark* umumnya memiliki distribusi data yang mirip dan dapat memuat sampel sintetis, label heuristik, atau label yang mengandung derau (*noise*) (Lu dan Lagaisse, 2025). Lu dan Lagaisse (2025) menemukan bahwa model yang dilatih pada satu dataset menggeneralisasi dengan buruk ke dataset dengan distribusi berbeda. Juliet milik NIST, misalnya, digambarkan sebagai dataset sintetis berukuran besar yang berisi potongan kode C/C++ berlabel (Lu dan Lagaisse, 2025).

II.3 *Explainable AI* untuk Model Kode

II.3.1 Konsep Dasar XAI

Model AI yang kompleks mengambil keputusan tanpa memberikan penjelasan yang jelas, yang dikenal sebagai masalah *black-box* (Arora dkk., 2025). *Explainable AI* (XAI) hadir untuk membuat sistem AI lebih mudah ditafsirkan dan transparan, sehingga pemangku kepentingan dapat mempercayai, memverifikasi, dan bertindak berdasarkan hasilnya (Arora dkk., 2025).

Pemahaman terhadap model dapat dicapai melalui dua jalur. Jalur pertama adalah memakai model yang secara inheren dapat ditafsirkan (*intrinsic*), yang juga disebut model *self-explaining* (Haji Mohammadkhani dkk., 2023). Model jenis ini sering menghadapi *trade-off* antara akurasi dan interpretabilitas (Haji Mohammadkhani dkk., 2023). Jalur kedua adalah pendekatan *post-hoc*, yang memperkirakan model *black-box* dengan model pengganti (*surrogate*) yang transparan tanpa mengubah model dasarnya (Haji Mohammadkhani dkk., 2023). LIME dan SHAP termasuk di antara metode *post-hoc* yang paling umum (Haji Mohammadkhani dkk., 2023).

Berdasarkan cakupannya, penjelasan dibedakan menjadi lokal dan global. Interpretabilitas lokal meninjau satu instans dan menjelaskan bagaimana prediksi untuk instans tersebut berkaitan dengan fitur-fiturnya (Haji Mohammadkhani dkk., 2023). Interpretabilitas global memahami bagaimana model mengambil keputusan berdasarkan seluruh fitur, parameter, dan strukturnya (Haji Mohammadkhani dkk., 2023).

Alat interpretabilitas yang dikaji pada model kode bekerja pada level *token* masukan (Steenhoek, Rahman, Sharmin, dan Le, 2023). Misalnya, Pintore, Piras, Sotgiu, Pintor, dan Biggio (2025) menghitung skor relevansi untuk setiap *token* pada tiap baris dengan skor *attention*, AttnLRP, atau *integrated gradients*. Akibatnya, penjelasan level *token* tidak dapat dibandingkan langsung dengan fitur *bug* yang ditandai pada level *token* AST (Steenhoek dkk., 2023), dan cara terbaik menghitung

cakupannya pada penjelasan berbasis sekuens masih menjadi pertanyaan terbuka (Cheng dkk., 2024). Masalah pemetaan ini dibahas pada Subbab II.4.

II.3.2 SHAP dan LIME

SHAP (*SHapley Additive exPlanations*) memberi setiap fitur sebuah nilai kepentingan untuk satu prediksi tertentu (Lundberg dan Lee, 2017). Nilai Shapley dihitung sebagai rata-rata berbobot dari selisih keluaran model untuk semua kemungkinan subset fitur (Lundberg dan Lee, 2017). Kelebihannya, Lundberg dan Lee (2017) menunjukkan bahwa pada kelas ukuran kepentingan fitur aditif terdapat satu solusi unik yang memiliki sejumlah properti yang diinginkan. Perhitungan klasik melibatkan semua subset fitur; untuk itu tersedia metode perkiraan seperti *Shapley sampling values* dan *Kernel SHAP*, dan metode yang terakhir memerlukan evaluasi model lebih sedikit untuk akurasi perkiraan yang serupa (Lundberg dan Lee, 2017).

LIME menjelaskan prediksi pengklasifikasi apa pun dengan mempelajari model yang dapat ditafsirkan secara lokal di sekitar prediksi tersebut (Ribeiro, Singh, dan Guestrin, 2016). Penjelasannya difokuskan pada model linear jarang (*sparse linear model*) yang dicari dengan perturbasi (Ribeiro dkk., 2016). LIME termasuk penjelasan *post-hoc* berbasis perturbasi yang banyak dipakai (Zhou, Hooker, dan Wang, 2021). Namun, metode berbasis perturbasi ini dilaporkan menunjukkan ketidakstabilan yang besar, yang mengganggu efektivitas metode dan kepercayaan pengguna (Zhou dkk., 2021). Ribeiro dkk. (2016) sendiri menyebut jumlah sampel yang tepat dan optimasi komputasi sebagai pekerjaan lanjutan.

II.3.3 *Attention* sebagai Penjelasan

Mekanisme *attention* menghasilkan distribusi bobot atas unit masukan yang diperhatikan model, dan distribusi ini sering dipandang, setidaknya secara implisit, sebagai petunjuk kepentingan relatif masukan (Jain dan Wallace, 2019). *Attention map* juga termasuk keluaran tambahan yang dapat dihasilkan model yang secara intrinsik dapat ditafsirkan (Haji Mohammadkhani dkk., 2023). Pada model kode, LineVul yang berbasis CodeBERT menghasilkan penjelasan level baris dari *self-attention* (Steenhoek dkk., 2023).

Jain dan Wallace (2019) menemukan bahwa bobot *attention* sering tidak berkorelasi dengan ukuran kepentingan fitur berbasis gradien, dan ada distribusi *attention* yang sangat berbeda tetapi menghasilkan prediksi yang setara. Mereka menyimpulkan bahwa modul *attention* standar tidak memberikan penjelasan yang bermakna dan tidak seharusnya diperlakukan seolah-olah demikian (Jain dan Wallace, 2019). Wiegreffe dan Pinter (2019) berpendapat bahwa Jain dan Wallace mengajukan pertanyaan yang penting, tetapi beberapa asumsi pada desain eksperimennya memberi kebebasan yang terlalu besar, sehingga praktisi tidak memiliki cara terapan untuk mengukur kegunaan distribusi *attention* pada kasus tertentu (Wiegreffe dan Pinter, 2019). Perdebatan ini menunjukkan bahwa nilai *attention* sebagai penjelasan perlu diuji, bukan diasumsikan.

II.4 Evaluasi Penjelasan XAI

II.4.1 Keterbatasan Metrik yang Ada untuk Domain Kode

II.5 Penelitian Terkait dan Celah Penelitian

II.5.1 Celah Penelitian

1. Belum ada *ground truth* formal berbasis *taint-flow*.
2. Belum ada fokus pada Java SQLi.
3. Belum ada perbandingan SHAP/LIME/*attention* dengan basis yang sama.

BAB III
ANALISIS MASALAH DAN RANCANGAN SOLUSI

Tujuan utama penulisan bab ini adalah untuk menguraikan masalah dan gambaran solusi penyelesaian masalah tugas akhir spesifik individu anggota tim capstone. Judul bab 3 dapat disesuaikan dengan konteks persoalan, misalnya: Analisis Masalah dan Rancangan Solusi Pembangunan Sistem ABC. Bab ini mencakup antara lain subbab-subbab:

III.1 Analisis Masalah

Deskripsi dan analisis dikaitkan dengan Rumusan Masalah tugas akhir, misalnya menjelaskan secara detail latar belakang dan masalah atau tantangan yang menjadi dasar perlunya solusi tugas akhir, menunjukkan *gap*/celah antara kondisi saat ini dengan kondisi yang diharapkan, serta dapat juga dijelaskan kaitan antara sistem/aplikasi yang dikembangkan dengan sistem/aplikasi lain yang terkait.

III.2 Gambaran Umum Solusi

Analisis solusi dapat terdiri dari pilihan alternatif solusi masalah tugas akhir untuk setiap permasalahan berdasarkan hasil studi literatur atau survei, pemilihan solusi beserta justifikasinya.

Deskripsi umum gambaran awal solusi tugas akhir yang dipilih, antara lain dapat mencakup:

- Gambaran awal modul/subsistem/komponen solusi yang akan dikembangkan untuk menyelesaikan masalah sebagai bagian dari solusi tugas akhir, berikut penjelasannya.
- Gambaran awal alur umum algoritma atau langkah-langkah pengembangan solusi tugas akhir dan penjelasannya.
- Penggunaan kakas secara umum yang diperlukan.

Untuk memperjelas gambaran solusi, dianjurkan menggunakan gambar diagram sebagai pendukung penjelasan bagian ini.

BAB IV
RENCANA PELAKSANAAN

Bab Rencana Pelaksanaan yang merupakan bab penutup Laporan Proposal Tugas Akhir digunakan untuk mendeskripsikan rencana pelaksanaan berupa jadwal dan risiko-risiko yang mungkin dihadapi dan rencana mitigasinya. Tujuan bab ini adalah:

- Mahasiswa memiliki rencana yang jelas mengenai pelaksanaan tugas akhir.
- Mahasiswa mengenali risiko-risiko pelaksanaan tugas akhir yang mungkin dihadapi dan sudah menyiapkan diri untuk mengantisipasi risiko tersebut.

IV.1 Jadwal

Cantumkan jadwal pengerjaan tugas akhir dan dilengkapi dengan uraiannya.

IV.2 Risiko

Cantumkan 5 risiko tertinggi yang mungkin dihadapi dalam pengerjaan tugas akhir. Risiko yang dicantumkan dapat merupakan risiko dari sisi teknis, risiko dari sisi operasional, risiko dari metode yang dipilih, dan sebagainya. Cantumkan pula rencana mitigasi dari risiko-risiko tersebut.

DAFTAR PUSTAKA

Arora, L., Girija, S. S., Kapoor, S., Raj, A., Pradhan, D., dan Shetgaonkar, A. (2025). Explainable artificial intelligence techniques for software development lifecycle: A phase-specific survey. Dalam *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)* (hal. 2281–2288). IEEE. doi: 10.1109/COMPSAC65507.2025.00321

Baker Effendi, S. D., Pinho, X., Dreyer, A. M., dan Yamaguchi, F. (2025). Scalable language agnostic taint tracking using explicit data dependencies. Dalam *Proceedings of the 14th ACM SIGPLAN International Workshop on the State of the Art in Program Analysis* (hal. 36–42). ACM. doi: 10.1145/3735544.3735586

Cheng, B., Zhao, S., Wang, K., Wang, M., Bai, G., Feng, R., … Wang, H. (2024). Beyond fidelity: Explaining vulnerability localization of learning-based detectors. *ACM Transactions on Software Engineering and Methodology*, *33*(5), 1–33. doi: 10.1145/3641543

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., … Zhou, M. (2020). CodeBERT: A pre-trained model for programming and natural languages. Dalam *Findings of the Association for Computational Linguistics: EMNLP 2020* (hal. 1536–1547). Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.139

Ganz, T., Rall, P., Härterich, M., dan Rieck, K. (2023). Hunting for truth: Analyzing explanation methods in learning-based vulnerability discovery. Dalam *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)* (hal. 524–541). IEEE. doi: 10.1109/EuroSP57164.2023.00038

Haji Mohammadkhani, A., Bommi, N. S., Daboussi, M., Sabnis, O., Tantithamthavorn, C., dan Hemmati, H. (2023). A systematic literature review of explainable AI for software engineering. *arXiv preprint arXiv:2302.06065*. doi: 10.48550/arXiv.2302.06065

Halfond, W. G. J., Viegas, J., dan Orso, A. (2006). A classification of SQL injection attacks and countermeasures. Dalam *Proceedings of the IEEE International Symposium on Secure Software Engineering*. Diakses dari `https://sites.cc.gatech.edu/home/orso/papers/halfond.viegas.orso.ISSSE06.pdf`

Jain, S., dan Wallace, B. C. (2019). Attention is not explanation. Dalam *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (hal. 3543–3556). Association for Computational Linguistics. doi: 10.18653/v1/N19-1357

Lu, C., dan Lagaisse, B. (2025). From lab to reality: A practical evaluation of deep learning models and LLMs for vulnerability detection. *arXiv preprint arXiv:2512.10485*. doi: 10.48550/arXiv.2512.10485

Lundberg, S. M., dan Lee, S.-I. (2017). A unified approach to interpreting model predictions. Dalam *31st Conference on Neural Information Processing Systems (NIPS 2017)*. Diakses dari `https://arxiv.org/abs/1705.07874`

Luo, Y., Xu, W., dan Xu, D. (2022). Compact abstract graphs for detecting code

vulnerability with GNN models. Dalam *Proceedings of the 38th Annual Computer Security Applications Conference* (hal. 497–507). ACM. doi: 10 .1145/3564625.3564655

Pintore, M., Piras, G., Sotgiu, A., Pintor, M., dan Biggio, B. (2025). Evaluating line-level localization ability of learning-based code vulnerability detection models. *arXiv preprint arXiv:2510.11202*. doi: 10.48550/arXiv.2510.11202

Ribeiro, M. T., Singh, S., dan Guestrin, C. (2016). "why should i trust you?": Explaining the predictions of any classifier. Dalam *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (hal. 1135–1144). ACM. doi: 10.1145/2939672.2939778

Steenhoek, B., Rahman, M. M., Sharmin, S., dan Le, W. (2023). Do language models learn semantics of code? a case study in vulnerability detection. *arXiv preprint arXiv:2311.04109*. doi: 10.48550/arXiv.2311.04109

Wiegreffe, S., dan Pinter, Y. (2019). Attention is not not explanation. Dalam *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (hal. 11–20). Association for Computational Linguistics. doi: 10.18653/v1/D19-1002

Yamaguchi, F., Golde, N., Arp, D., dan Rieck, K. (2014). Modeling and discovering vulnerabilities with code property graphs. Dalam *2014 IEEE Symposium on Security and Privacy* (hal. 590–604). IEEE. doi: 10.1109/SP.2014.44

Yamaguchi, F., Maier, A., Gascon, H., dan Rieck, K. (2015). Automatic inference of search patterns for taint-style vulnerabilities. Dalam *2015 IEEE Symposium on Security and Privacy* (hal. 797–812). IEEE. doi: 10.1109/SP.2015.54

Zhou, Z., Hooker, G., dan Wang, F. (2021). S-LIME: Stabilized-LIME for model explanation. Dalam *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining* (hal. 2429–2438). ACM. doi: 10.1145/ 3447548.3467274

Lampiran A. Contoh Judul Lampiran

Untuk TA Capstone diwajibkan melampirkan dokumen kelengkapan proyek capstone antara lain:

- Rencana Umum Proyek
- Spesifikasi Kebutuhan Sistem
- Rancangan Sistem
- Pengujian Sistem (terutama *Integration Testing*)

A.1 Contoh Judul Anak Lampiran

Contoh anak lampiran.